

Devops Assignment

Assignment: The "Release Manager" Roleplay

Course: SDLC & DevOps Fundamentals

Project: The "Smart-Attend" Mobile App

Task 1: Waterfall Model Failure Analysis for the Smart-Attend Application

Introduction:

The Smart Attend Mobile App is a software system aimed at automating student attendance using Geo-fencing, a technology that automatically records attendance when a student enters a classroom Geo-fence. During the software development lifecycle, a significant change in requirements was introduced by the university administration: the implementation of a Biometric Face ID system, which will ensure that no proxy attendance is allowed.

If the software development lifecycle adopted for the project is the Waterfall SDLC model, then a number of challenges will arise in the development of the software system, as discussed in the following sections of this chapter. In the Waterfall SDLC model, software

development occurs in a sequential manner, i.e., each phase must be completed before moving on to the next phase in the waterfall model. Because of its inflexibility, it is extremely difficult to incorporate a change in requirements in a software system in its later stages of development lifecycle.

1. Rigidity of Sequential Phases :

The Waterfall model divides software development into distinct phases:

- 1. Requirements Analysis**
- 2. System Design**
- 3. Implementing**
- 4. Testing**
- 5. Deployment**
- 6. Maintenance**

In this model, each phase must be completed and finalized before moving to the next stage. Once the project progresses beyond the requirements phase, it becomes extremely difficult to revisit and modify the initial specifications.

In the Smart-Attend project, the original requirement was limited to:

- >Student login authentication
- >Geo-fencing based location detection
- >Automatic attendance marking

However, the new requirement introduces biometric Face ID authentication, which significantly changes the system architecture. Implementing this feature requires:

- Camera hardware access
- Image processing capabilities
- Face recognition algorithms
- Biometric security modules
- Identity verification workflows

Since the system design and development phases would already be in progress, the team would need to return to the requirement and design phases, causing disruption in the development pipeline. The Waterfall model does not support iterative requirement changes effectively, leading to delays and rework.

2. High Cost of Requirement Changes

One of the biggest limitations of the Waterfall model is the high cost associated with late requirement changes. In traditional development environments, system architecture is designed based on the initial specifications. When new requirements emerge later in the project, significant modifications are required across multiple components.

In the Smart-Attend system, integrating Face ID verification would require the following architectural changes:

- Integration of biometric authentication libraries
- Modification of the authentication module
- Addition of camera access APIs

- Implementation of image processing and face recognition algorithms
- Storage and encryption of biometric data
- Updates to the database schema for biometric records

These changes impact multiple layers of the application architecture:

- Frontend layer – Camera interface and biometric verification UI
- Backend layer – Face recognition logic and authentication services
- Database layer – Secure storage of biometric identifiers
- Security layer – Encryption and identity verification mechanisms

Because these modifications occur after the design phase is finalized, developers would need to redesign system modules and rewrite portions of existing code, which increases development cost, time, and complexity.

3. Delayed Testing and Integration Issues

In the Waterfall model, testing is typically performed after the entire development phase is completed. This means that errors, compatibility problems, and integration issues are discovered only at the end of the project lifecycle.

In the Smart-Attend application, integrating Face ID verification introduces multiple new dependencies and technical risks, such as:

- Camera permission management on mobile devices
- Accuracy of face recognition algorithms
- Environmental factors affecting biometric recognition
- Security vulnerabilities related to biometric authentication
- Integration with existing geo-fencing logic

If these components are tested only at the final testing phase, the development team may encounter major system failures or performance issues. Fixing such issues late in the development cycle requires extensive debugging, redesign, and redevelopment.

As a result, the project timeline may be significantly delayed, and the final product may not meet the expected release schedule.

4. Lack of Early User Feedback

Another limitation of the Waterfall model is the absence of continuous user feedback during development. The working software is usually delivered only at the end of the project lifecycle.

In the case of the Smart-Attend system, early user feedback from stakeholders such as:

- Students
- Faculty members
- University administrators

could help identify usability issues, security concerns, and performance problems related to the attendance system.

However, because the Waterfall model does not deliver incremental releases, stakeholders would only evaluate the system after full completion. If the biometric authentication feature introduces usability challenges or technical limitations, correcting these problems would require major post-development modifications.

Conclusion

The Waterfall model is not suitable for projects where requirements are likely to evolve during development. In the Smart-Attend application, the sudden introduction of biometric Face ID verification significantly alters the system architecture and functionality.

Due to the rigid sequential structure of the Waterfall model, this change would lead to:

- Significant project delays
- Increased development cost
- Complex system redesign
- Late detection of integration issues

Therefore, adopting a flexible and iterative development methodology such as Agile (Scrum) would be more appropriate. Agile allows teams to incorporate changing requirements, deliver incremental features through short development cycles, and continuously improve the product based on stakeholder feedback.

Task 2: Agile Pivot Using the Scrum Framework

Introduction:

i) After realizing the limitations of the Waterfall model, the Smart-Attend mobile application project development team decides to use the Agile software development methodology, specifically the Scrum framework. Agile methodology emphasizes the use of iterative software development, so it is very applicable to software development projects where the project requirements change during the software development life cycle.

li) The Scrum framework of the Agile methodology divides the software development process into iterations, called Sprints, which last 1 to 2 weeks. Each Sprint iteration produces a fully functional software increment, so the project development team can easily respond to changes and incorporate the changes and feedback from the stakeholders of the project.

The Smart-Attend mobile application project development team, by using the Scrum framework of the Agile methodology, can gradually develop the system and incorporate the new Biometric Face ID project requirement.

Scrum Roles in the Smart-Attend Project:

In Scrum, the project team is divided into three primary roles:

1. Product Owner

The Product Owner represents the stakeholders and is responsible for managing the Product Backlog, which contains all required features and system improvements. In the Smart-Attend project, the Product Owner ensures that features such as geo-fencing, attendance tracking,

and biometric verification align with the university's requirements.

2. Scrum Master

The Scrum Master acts as a facilitator who ensures that the Scrum process is followed correctly. The Scrum Master removes obstacles, improves team collaboration, and ensures smooth Sprint execution.

3. Development Team

The Development Team consists of software engineers, UI developers, backend developers, and QA testers who work together to design, implement, and test the Smart-Attend application during each Sprint.

Sprint Planning for the Smart-Attend Application:

To manage the new biometric requirement efficiently, the project is divided into two 2-week Sprints, each focusing on specific deliverables.

Sprint 1: Minimum Viable Product (MVP)

Objective

The goal of Sprint 1 is to develop the Minimum Viable Product (MVP) of the Smart-Attend system. The MVP represents the basic functional version of the application that can perform the core attendance functionality using geo-fencing.

Key Features Implemented

User Authentication System

Students and faculty members can securely log into the application using their university credentials.

Basic User Interface (UI)

Development of the primary mobile application interface including login pages, dashboard screens, and attendance notifications.

Geo-fencing Based Attendance Detection

Implementation of location-based detection logic that determines whether a student is within the defined classroom radius (e.g., within 10 meters).

Real-Time Attendance Monitoring

Faculty members can view a real-time list of students who are present in the classroom based on geo-fencing data.

Outcome of Sprint 1

At the end of Sprint 1, the development team delivers a working prototype where students can log in and have their attendance automatically recorded when they enter the classroom area.

This early release allows stakeholders to:

- Test the system functionality
- Provide feedback on usability
- Identify performance issues early

Sprint 2: Biometric Integration and System Enhancement

Objective

The goal of Sprint 2 is to integrate the Biometric Face ID verification system into the existing Smart-Attend application and enhance system features based on feedback received from Sprint 1.

Key Features Implemented

Face ID Biometric Authentication

A facial recognition module is integrated into the system to verify the identity of students before marking attendance. This prevents proxy attendance and improves system security.

Camera Access and Image Processing

The application requests camera permissions and captures facial images for verification using biometric recognition algorithms.

Biometric Verification Workflow

The attendance marking process is updated as follows:

Student enters classroom → Geo-fencing detects location → Face ID verification → Attendance confirmed.

Reporting Dashboard Improvements

The faculty dashboard is enhanced to display:

- Attendance reports
- Student verification status
- Attendance analytics

Bug Fixes and Performance Optimization

Feedback collected from Sprint 1 testing is used to fix bugs, improve system performance, and refine the user experience.

Outcome of Sprint 2

At the end of Sprint 2, the Smart-Attend system becomes a secure and reliable attendance platform that combines location tracking and biometric verification. The system is now capable of preventing proxy attendance while maintaining an efficient automated attendance process.

Advantages of Using Scrum in the Smart-Attend Project

Adopting the Scrum framework provides several benefits for the Smart-Attend development process:

Flexibility in Handling Changes

Scrum allows new features, such as Face ID verification, to be added in upcoming Sprints without restarting the entire development process.

Faster Feature Delivery

Each Sprint delivers a working product increment, enabling faster deployment of new features.

Continuous Testing and Feedback

Regular Sprint reviews and testing cycles help detect technical issues early and improve product quality.

Improved Collaboration

Frequent meetings such as Daily Scrum, Sprint Planning, Sprint Review, and Sprint Retrospective ensure effective communication among team members and stakeholders.

Conclusion

The adoption of the Scrum framework enables the Smart-Attend development team to respond effectively to changing requirements while maintaining development efficiency. By dividing the project into iterative Sprints, the team can first deliver the core geo-fencing functionality and later integrate advanced features such as biometric Face ID verification.

Task 3: Sprint Backlog for Sprint 1

Introduction

The Sprint Backlog refers to a set of items from the Product Backlog that the development team commits to completing within a particular Sprint. The items are created during the Sprint Planning meeting and are a reflection of the work to be completed to develop a functional product increment.

For the Smart-Attend Mobile Application, the items to be completed within the first Sprint are related to developing the Minimum Viable Product (MVP). The main aim of this Sprint will be to develop the basic functionality for automated attendance based on Geo-fencing, basic user authentication, and interface elements.

The items within the Sprint Backlog are represented as User Stories, which are written as follows:

"As a [user], I want [feature], so that [benefit]."

Task ID	User Story	Priority	Estimated Hours	Assigned Role	Status
US01	As a student, I want to login using my university ID so that I can securely access my profile and attendance dashboard.	High	4	Backend Developer	Pending

Task ID	User Story	Priority	Estimated Hours	Assigned Role	Status
US02	As a teacher, I want to view a real-time list of present students so that I can monitor attendance instantly.	High	6	Frontend Developer	Pending
US03	As a system, I want to detect if a student is within a 10- meter radius of the classroom so that attendance can be marked automatically.	High	8	Backend Developer	Pending
US04	As a student, I want to receive a notification when my attendance is successfully recorded so that I can confirm it was marked.	Medium	3	Mobile Developer	Pending

Task ID	User Story	Priority	Estimated Hours	Assigned Role	Status
US05	As an administrator, I want the ability to manually override attendance records so that technical errors can be corrected.	Low	4	Backend Developer	Pending

Explanation of the Tasks in the Sprint Backlog:

US01 – Student Login Authentication

The main aim of this task is to provide a secure authentication mechanism for the students of the university. The application will provide the students with the ability to login using their university credentials.

The main components of this task include:

- The login interface of the application.
- The API authentication endpoint.
- The session management mechanism.

US02 – Real-Time Attendance Monitoring

This feature allows the teachers of the university to monitor the attendance of the students in real-time using the application. The application will update the attendance in real-time as the students enter the classroom area.

The technologies that may be used in the implementation of this feature include:

- The application of the WebSocket or real-time API.
- The application of the dynamic UI update mechanism.

US03 – Geo-Fencing Detection System

This is the main functionality of the Smart-Attend system. The application will detect whether the student is within the geographical boundary (10 meters) of the classroom location.

Main technical components:

- GPS location tracking
- Geo-fencing algorithm
- Distance calculation logic
- Attendance trigger system

US04 – Attendance Notification System

This system will ensure transparency on the part of the student, as they will be notified instantaneously once their attendance is marked.

Possible implementations:

- Mobile push notifications
- In-app alert messages

US05 – Manual Attendance Override

The US05 feature enables the user to manually override the attendance system in the case of technical failures, for instance:

- Failure of GPS
- Failure of network connectivity
- Failure of device permissions

The US05 feature ensures the reliability of the attendance system.

Sprint Goal

The objective of the first Sprint, i.e., Sprint 1, is to create the Minimum Viable Product (MVP) of the Smart Attend system, which would enable the user to track the attendance of the employees using the geo-fencing technique, along with basic authentication and monitoring capabilities.

The prototype of the attendance system would be developed by the end of the Sprint.